

LIVE FAKE NEWS DETECTION ON REDDIT

Project title: Live Fake News Detection on Reddit

Domain: Gaming

Document type: System Design Document

Prepared by:

Pooja Rao V

Sidharth KS

Rohith M

Prajwal KP

Ravitheja

Sakshith G N

Subhash S

Preetham A R

Venkatesh

Date of Submission: 28/11/2025

1. INTRODUCTION

1.1 Purpose of the Document

This design document defines the **architecture, components, data flow, and technical decisions** behind the LIVEFAKENEWS system.

It will serve as:

- A technical reference for developers
- A documentation artifact for academic evaluation
- A blueprint for future extensions

2. SYSTEM OVERVIEW

LIVEFAKENEWS is a **hybrid AI system** designed to:

1. Extract factual claims from Reddit posts
2. Search the real web for supporting evidence
3. Retrieve and rank evidence using semantic search
4. Verify claims using cross-encoder truth matching
5. Output: **TRUE / FALSE / UNVERIFIED**
6. Provide reasoning + citations

This system focuses on the **gaming domain** during Phase 1.

3. HIGH-LEVEL ARCHITECTURE

3.1 Architecture Diagram (Textual)

Reddit Post → Claim Extractor



Google Search API



Web Scraper (HTML → Text)



Sentence Transformer Embedder



FAISS Index



Cross-Encoder Reranker



Evidence Aggregation Logic



Final Output

(TRUE / FALSE / UNVERIFIED + Explanation + Citations)



4. SYSTEM COMPONENTS

4.1 Claim Extraction Module

File: factcheck/claim_extractor.py

Purpose: Convert Reddit title + body into a short factual claim.

Responsibilities:

- Extract first-sentence claims
- Remove fluff
- Produce a concise claim string

Example:

Input:

Title: "GTA 6 trailer leaked"

Body: "The first gameplay footage was uploaded today."

Output:

"GTA 6 trailer leaked. The first gameplay footage was uploaded today."

4.2 Domain Classifier (Rumor Likelihood)

Folder: model/domain_classifier/

Model: DeBERTa-v3-base fine-tuned

Labels: true, false, unverified

Purpose:

 **Not truth verification.**

It estimates how "rumor-like" the text is based on language patterns in the gaming domain.

Used only as a **signal** in the final decision.

4.3 Search Module

File: factcheck/search.py

Uses **SerpAPI** to find top web results.

Output:

```
[  
  { "title": "...", "url": "..." },  
  ...  
]
```

Rationale:

- Google Search gives best coverage
- SerpAPI avoids scraping Google directly

4.4 Scraper Module

File: factcheck/scrapper.py

Fetches HTML and extracts paragraphs using BeautifulSoup.

Requirements:

- Only text extraction
- No images, JS, scripts
- Safe truncation to 50k chars

4.5 Embedding & Retrieval Module

File: factcheck/retriever.py

Model: sentence-transformers/all-MiniLM-L6-v2

Steps:

1. Embed all pages
2. Build FAISS index
3. Retrieve top-k relevant evidence

Why MiniLM?

Lightweight, fast, high semantic accuracy.

4.6 Cross-Encoder Reranker

File: factcheck/reranker.py

Model: cross-encoder/ms-marco-MiniLM-L-6-v2

Purpose:

The cross-encoder compares:

[claim] + [evidence text]

and outputs a **relevance/entailment score**.

Reason:

Cross-encoders are much more accurate than bi-encoders.

4.7 Evidence Aggregation Module

File: factcheck/aggregator.py

Inputs:

- Classifier result
- Top reranked evidence

Logic:

avg_score > 1.2 → TRUE

avg_score < 0.3 → FALSE

else → UNVERIFIED

Output:

- Final label
- Explanation
- Citations
- Evidence confidence score

4.8 Fact-Checking Pipeline

File: factcheck/pipeline.py

Executes full workflow:

- Claim extraction
- Search → Scrape
- Embed → Retrieve
- Rerank
- Aggregate

Output JSON:

```
{  
  "claim": "...",  
  "classifier": {...},  
  "evidence": [...],  
  "final": {  
    "final_label": "true/false/unverified",  
    "reason": "..."  
  }  
}
```

4.9 FastAPI Deployment

File: app/api.py

Endpoint:

POST /factcheck

Request:

```
{  
  "title": "...",  
  "selftext": "..."  
}
```


Response:

Full fact-check JSON.

5. DATA FLOW DESIGN

5.1 Text Flow

1. Reddit text → Claim extractor
2. Embedding → Vector index
3. Evidence → Reranker
4. Labels → Aggregator

5.2 Model Flow

- DeBERTa → rumor score
- MiniLM → semantic similarity
- CrossEncoder → truth inference

6. ALGORITHMIC DESIGN

6.1 Hybrid Verification Model

Stage 1: Language Classification

Estimates how “fake-news-like” the wording is.

Stage 2: Retrieval-Based Verification

Uses passages from the web.

Stage 3: Cross-Encoder Truth Estimation

Pairs Claim × Evidence.

Stage 4: Rule-Based Aggregation

Weighted decision logic.

7. NON-FUNCTIONAL REQUIREMENTS

Performance

- < 3 seconds per fact-check for cached pages
- SerpAPI latency dominates (~1.5 sec avg)

Scalability

- FAISS scales to millions of documents
- FastAPI horizontally scalable

Maintainability

- Modular folder structure
- Easy to swap models
- Hot-swappable retriever/encoder

Security

- No storing user posts
- Scrapes only public web pages

8. TECHNOLOGY STACK

Backend

- **Python 3.10**
- **FastAPI**
- **Uvicorn**

ML Models

- HuggingFace Transformers
- SentenceTransformers
- FAISS

External APIs

- SerpAPI
- Reddit API (future)

9. RISKS & LIMITATIONS

Risk	Severity	Mitigation
Web misinformation sources	High	Trust scoring + domain whitelisting
SerpAPI rate limits	Medium	Add backups (Bing API)
Scraper failure	Medium	Retry + fallback logic
Lingering bias in classifier	Low	Larger fine-tuning dataset

10. FUTURE EXTENSIONS

- Live Reddit bot replying in comments
- Full UI dashboard
- Abstractive summarization of evidence
- OpenAI or Llama model for reasoning
- Model distillation for speed
- Database for caching results

11. CONCLUSION

This design document outlines the full technical architecture required to build a **production-ready fact-checking system** specialized for Reddit gaming misinformation.

The system integrates:

- Classical retrieval
- Modern LLM transformers
- Cross-encoder truth inference
- Automated web search

- Evidence-based rule logic